

B.TECH – COMPUTER SCIENCE

Data Structures & Algorithms

UNIT 1 – Introduction to Data Structures, Arrays, Structures, Pointers, Dynamic Memory Allocation & Complexity Analysis

A Complete Beginner-Friendly Guide

What We Will Cover Today

01

Introduction to Data Structures

Definition, need, classification & operations

02

Arrays, Structures & Unions

Review of arrays, structures, self-referential structures, unions

03

Pointers & Dynamic Memory

Pointers, malloc/calloc/realloc/free

04

Arrays in Memory

Linear array representation & dynamically allocated arrays

05

Performance Analysis

Time complexity, space complexity, Big-O notation

01

Introduction to Data Structures

Understanding what data structures are and why they matter

What is a Data Structure?

- **Definition**

- A Data Structure is a specialized way of organizing, storing and processing data in a computer so that it can be accessed and used efficiently.

- **The Formula**

- Data Structure = Organized Data + Relationships + Set of Operations

- **In Simple Words**

- It is a container that holds data in a particular layout (like a row, a chain, or a tree) so operations like search, insert and delete can be done quickly.

- **Common Examples**

- Array, Linked List, Stack, Queue, Tree, Graph, Hash Table



Key Idea

- Data alone is just raw values (e.g. 10, 20, 30)
- A data structure adds meaning — how those values relate and how we work with them
- Choosing the right data structure = faster, more efficient programs

Why Do We Need Data Structures?

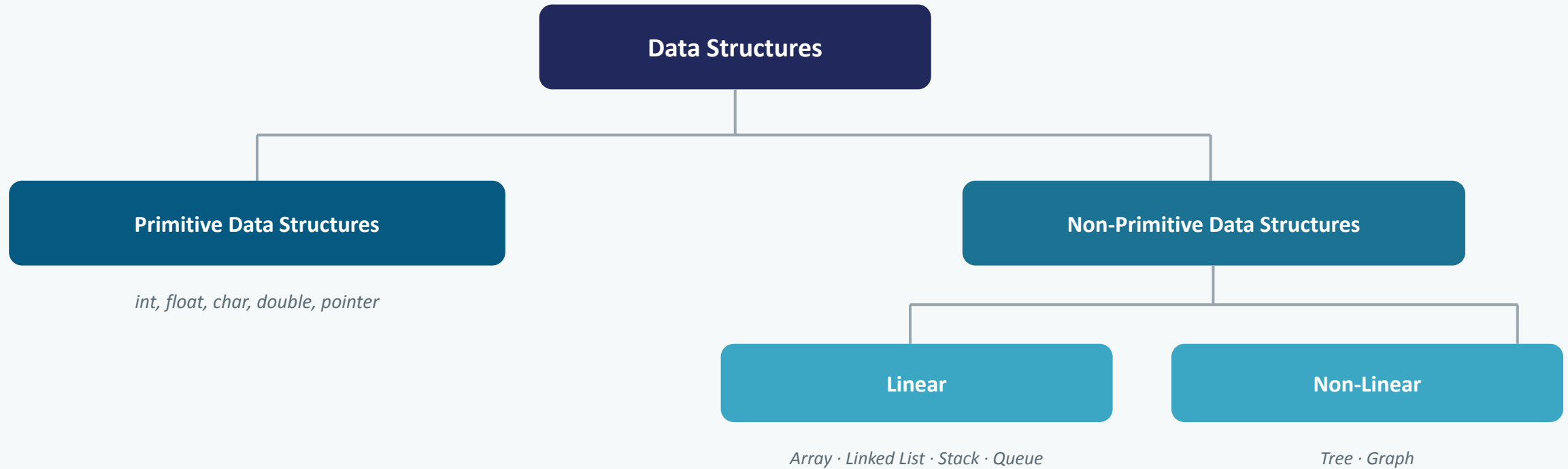
- Efficient Storage — organize large volumes of data using minimum memory
- Efficient Processing — perform search, insert, delete and update operations quickly
- Reusability — once designed, a data structure can be reused across many programs
- Abstraction — hides implementation details and exposes only necessary operations
- Better Algorithm Design — many algorithms are built directly on top of specific data structures
- Real-World Applications — used in databases, operating systems, compilers, AI, networking, and every major software system



Without DS...

- Searching becomes slow (checking one item at a time)
- Memory gets wasted or mismanaged
- Programs cannot scale to handle large real-world data

Classification of Data Structures



Primitive: Basic data types directly operated upon by machine-level instructions.

Non-Primitive: Derived from primitive types; further divided into Linear (sequential arrangement) and Non-Linear (hierarchical/networked arrangement).

Primitive Data Structures

- **Definition**

- Basic data types that are directly operated upon by machine-level instructions. They are the building blocks of all other data structures.

- **Characteristics**

- Predefined by the programming language
- Cannot be broken down further
- Stored directly in a single memory location

- **Examples**

- int (integers), float / double (decimal numbers), char (single characters), boolean (true/false), pointer (memory addresses)



Memory Size (typical)

- char → 1 byte
- int → 4 bytes
- float → 4 bytes
- double → 8 bytes
- pointer → 4/8 bytes

Non-Primitive Data Structures

- **Definition**

- Data structures that are derived from primitive data types and can hold multiple values with defined relationships between them.

- **Linear Data Structures**

- Elements are arranged in a sequential order, one after another. Every element (except first & last) has a unique predecessor and successor. Example: Array, Linked List, Stack, Queue.

- **Non-Linear Data Structures**

- Elements are NOT arranged sequentially — each element may connect to multiple other elements, forming hierarchical or networked relationships. Example: Tree, Graph.



Quick Test

- Can you draw it as a single straight line? → Linear
- Does it branch out or form a network? → Non-Linear

Linear vs Non-Linear — Comparison

Basis	Linear Data Structure	Non-Linear Data Structure
Arrangement	Elements arranged sequentially	Elements arranged hierarchically / in a network
Levels	Single level involved	Multiple levels involved
Traversal	Can be traversed in a single run	Cannot be traversed in a single run
Memory	Usually easy, may waste memory (arrays)	Memory utilized efficiently
Complexity	Simple to implement	Comparatively complex to implement
Examples	Array, Stack, Queue, Linked List	Tree, Graph

Data Structure Operations

- **Traversing**
 - Visiting / accessing each element of the data structure exactly once to process it.
- **Insertion**
 - Adding a new element into the data structure.
- **Deletion**
 - Removing an existing element from the data structure.
- **Searching**
 - Finding the location of a particular element that satisfies given conditions.
- **Sorting**
 - Arranging elements in ascending or descending order.
- **Merging**
 - Combining elements of two similar data structures to form a single new data structure.



Remember

- Every algorithm you write is essentially a combination of these six basic operations
- Different data structures perform these operations with different efficiency

02

Arrays, Structures & Unions

Reviewing arrays and exploring structures, self-referential structures & unions

Review of Arrays

- **Definition**

- A collection of elements of the SAME data type, stored in contiguous (continuous) memory locations, accessed using an index.

- **Key Characteristics**

- Fixed size — declared once, size cannot change (static array)
- Homogeneous — all elements share the same data type
- Indexing starts from 0 in C / C++ / Java
- Random Access — any element can be accessed directly using its index in $O(1)$ time

- **Declaration Syntax**

- datatype array_name[size]; → example: int marks[5];

Advantages

- Fast random access
- Easy to implement & traverse
- Memory efficient (no extra pointers)

Types of Arrays

Type	Description	Declaration Example
One-Dimensional (1-D)	A simple linear list of elements	int arr[5];
Two-Dimensional (2-D)	Elements arranged as rows & columns (matrix)	int arr[3][4];
Multi-Dimensional	Array with more than 2 dimensions	int arr[2][3][4];

Structures in C

```
struct Student {  
    char name[20];  
    int  roll_no;  
    float marks;  
};  
  
int main() {  
    struct Student s1;  
    strcpy(s1.name, "Riya");  
    s1.roll_no = 12;  
    s1.marks   = 89.5;  
}
```



Explanation

- A structure is a user-defined data type that groups related variables of DIFFERENT data types under one name.
- Declared using the struct keyword.
- Each variable inside is called a member/field.
- Members are accessed using the dot (.) operator.
- Each member gets its own separate memory — total size = sum of all members.

Nested Structures & Array of Structures

- **Nested Structure**

- A structure that contains another structure as one of its members. Useful for representing composite real-world entities (e.g. an Employee structure containing a Date-of-Birth structure).
- `struct Date { int d, m, y; };`
- `struct Employee { char name[20]; struct Date dob; };`

- **Array of Structures**

- A collection of structure variables, where each element of the array is itself a complete structure. Useful for storing records of many entities (e.g. 50 students).
- `struct Student list[50];`
- `list[0].roll_no = 1;`



Why Useful?

- Models real-world records naturally (student, employee, product)
- Forms the foundation for databases and file records

Self-Referential Structures

```
struct Node {  
    int data;  
    struct Node *next;  
};  
  
// creating & linking nodes  
struct Node n1, n2;  
n1.data = 10;  
n1.next = &n2;  
n2.data = 20;  
n2.next = NULL;
```



Why It Matters

- A self-referential structure is a structure that contains a pointer to another structure of the SAME type as one of its members.
- It does NOT contain a copy of itself — only a pointer/address to another instance.
- This is the foundation for building dynamic data structures like Linked Lists, Stacks, Queues and Trees.
- The 'next' pointer links one node to another, forming a chain.

Unions in C

```
union Data {  
    int    i;  
    float  f;  
    char   c;  
};  
  
int main() {  
    union Data d;  
    d.i = 10;        // uses shared memory  
    d.f = 3.14;      // overwrites d.i  
    printf("%f", d.f);  
}
```



Explanation

- A union is a user-defined data type, declared using the union keyword, where ALL members share the SAME memory location.
- Size of a union = size of its LARGEST member (not the sum).
- Only ONE member holds a valid value at any given time — writing to one member overwrites the others.
- Used when memory is limited and only one of several types is needed at a time.

Structure vs Union

Basis	Structure (struct)	Union (union)
Memory Allocation	Separate memory for each member	Shared / common memory for all members
Total Size	Sum of size of all members	Size of the largest member
Access	All members can be accessed anytime	Only one member is valid at a time
Memory Efficiency	Uses more memory	Uses less memory
Use Case	Storing multiple related values together	Storing one of several possible value types

03

Pointers & Dynamic Memory Allocation

Working directly with memory addresses at runtime

Pointers — The Basics

- **Definition**

- A pointer is a variable that stores the memory address of another variable, instead of storing a data value directly.

- **Declaration Syntax**

- `datatype *pointer_name;` → example: `int *p;`

- **Two Key Operators**

- `&` (Address-of operator) — returns the memory address of a variable
- `*` (Dereference / Value-at operator) — accesses the value stored at the address

- **Why Pointers Are Important**

- They enable dynamic memory allocation, efficient array/function handling, and are essential for building linked data structures.



Example

- `int x = 10;`
- `int *p = &x;`
- `printf("%d", *p);` // prints 10
- `*p` now refers to x's value

Pointer Arithmetic & Pointers with Arrays

- **Pointer Arithmetic**

- Pointers can be incremented (p++) or decremented (p--). Moving a pointer by 1 actually moves it forward by the SIZE of its data type in bytes (not just 1 byte).

- **Pointers and Arrays**

- The name of an array is itself a pointer constant to its first element. So arr and &arr[0] point to the same address.
- *(arr + i) is exactly equivalent to arr[i]

- **Types of Pointers**

- NULL pointer, Void pointer (generic), Wild pointer (uninitialized — dangerous), Dangling pointer (points to freed memory)



Caution

- Always initialize pointers before use
- Avoid dangling pointers by setting freed pointers to NULL

Dynamic Memory Allocation Functions (stdlib.h)

Function	Purpose	Syntax
malloc()	Allocates a single block of requested memory (uninitialized)	<code>ptr = (type*) malloc(n * sizeof(type));</code>
calloc()	Allocates multiple blocks & initializes all bytes to zero	<code>ptr = (type*) calloc(n, sizeof(type));</code>
realloc()	Resizes a previously allocated memory block	<code>ptr = (type*) realloc(ptr, newSize);</code>
free()	Releases / de-allocates previously allocated memory	<code>free(ptr);</code>

Dynamic Memory Allocation — Example

```
#include <stdlib.h>

int n = 5, i;
int *arr;

// allocate memory at run time
arr = (int*) malloc(n * sizeof(int));

for (i = 0; i < n; i++)
    arr[i] = i * 10;

free(arr);    // release memory
```



Explanation

- Unlike static arrays, dynamic memory is allocated at RUN TIME, so the size can depend on user input.
- malloc() returns a void pointer, which is type-cast to the required data type.
- sizeof(type) ensures the correct number of bytes is reserved.
- free() MUST be called to release memory and avoid memory leaks.

04

Arrays in Memory

How linear arrays are actually stored & addressed in memory

Representation of Linear Arrays in Memory

A linear array is stored in a **contiguous (continuous) block** of memory locations, so the address of any element can be calculated directly — without traversing the array.

Address Formula for a 1-D Array

$$\text{LOC}(A[I]) = \text{Base}(A) + W \times (I - \text{Lower_Bound})$$

Base (A)

Address of the first element of the array

W

Size (in bytes) of one element / word size

I

Index of the element whose address is required

Lower_Bound

Smallest / starting index of the array (usually 0)

For 2-D arrays: Row-Major Order $\rightarrow \text{LOC}(A[I][J]) = \text{Base}(A) + W \times [(I - \text{LB}_r) \times N + (J - \text{LB}_c)]$ • Column-Major Order uses the reverse ordering of I and J.

Address Calculation — Worked Example

Given:

An array $A[1 \dots 10]$ is stored with $\text{Base}(A) = 1000$ and each element occupies $W = 2$ bytes. Find the address of $A[6]$.

$$\text{LOC}(A[I]) = \text{Base}(A) + W \times (I - \text{Lower_Bound})$$

$$\text{LOC}(A[6]) = 1000 + 2 \times (6 - 1)$$

$$\text{LOC}(A[6]) = 1000 + 2 \times 5$$

$$\text{LOC}(A[6]) = 1000 + 10$$

$$\text{LOC}(A[6]) = 1010$$



Result

The 6th element of the array is stored at memory address 1010.

This formula allows DIRECT (random) access to any array element in constant time — $O(1)$ — without scanning from the start.

Dynamically Allocated Arrays

```
int n;
printf("Enter size: ");
scanf("%d", &n);

// size decided at RUN TIME
int *arr = (int*) malloc(n * sizeof(int));

for (int i = 0; i < n; i++)
    arr[i] = i + 1;

free(arr);
```



Static vs Dynamic

- A dynamically allocated array is an array whose SIZE is decided at run time using functions like `malloc()` or `calloc()`, instead of being fixed at compile time.
- It still occupies a contiguous block of memory, so the same addressing formula applies.
- Advantage: memory is used efficiently — only as much as actually needed is allocated.
- Must be released manually using `free()` once no longer needed.

05

Performance Analysis of Algorithms

Measuring how good an algorithm really is — Time & Space Complexity

Performance Analysis of an Algorithm

- **Definition**

- Performance analysis is the process of determining the amount of computing resources (time and space/memory) an algorithm needs to execute.

- **Two Approaches**

- Priori Analysis — Theoretical estimation done BEFORE running the program (independent of the machine/compiler)
- Posteriori Analysis — Actual measurement done by RUNNING the program on a real machine (depends on hardware, compiler etc.)

- **Why Analyze Performance?**

- To compare multiple algorithms solving the same problem and choose the most efficient one, especially for large inputs.



Two Key Metrics

- Time Complexity — how execution time grows with input size
- Space Complexity — how memory usage grows with input size

Time Complexity & Asymptotic Notations

- **Time Complexity**

- The amount of time an algorithm takes to run, expressed as a function of the size of its input, n .

- **Big-O Notation — $O(f(n))$**

- Describes the WORST-case / upper bound on running time.

- **Omega Notation — $\Omega(f(n))$**

- Describes the BEST-case / lower bound on running time.

- **Theta Notation — $\Theta(f(n))$**

- Describes the AVERAGE-case / tight bound — when best and worst case grow at the same rate.

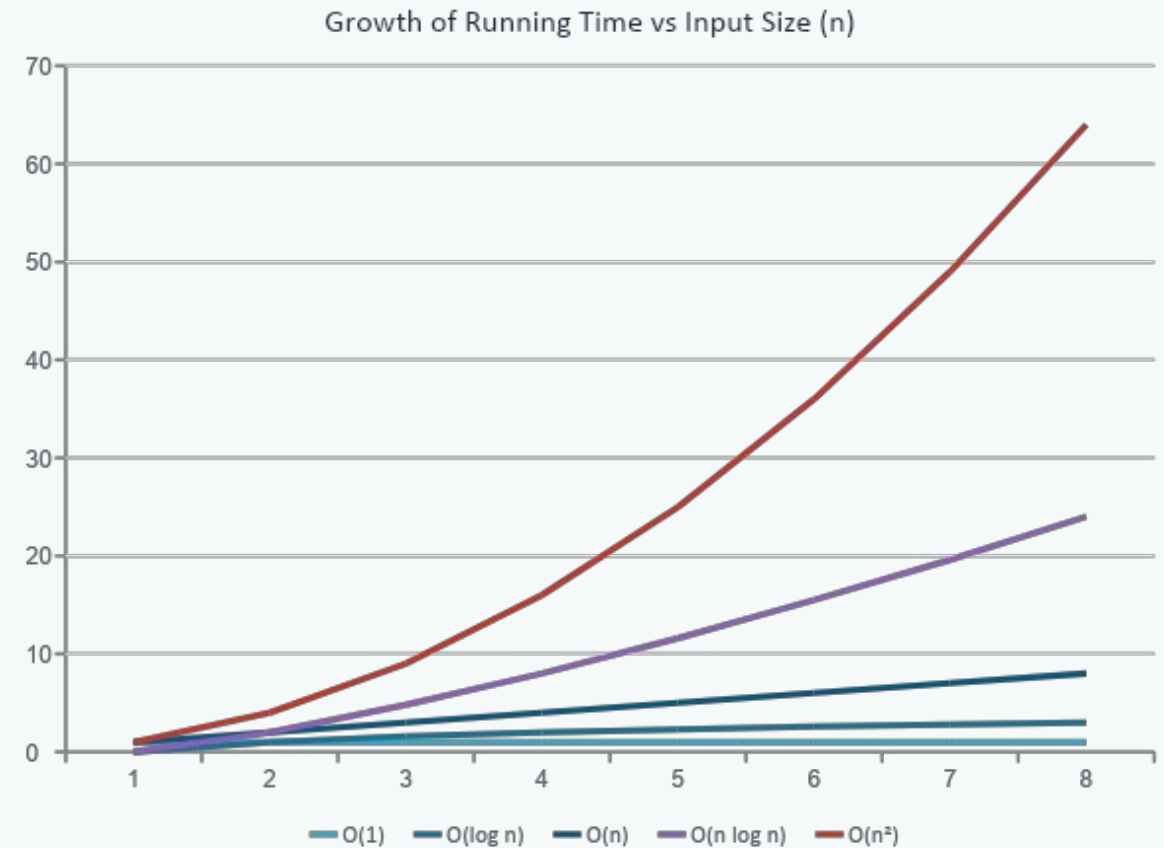


Most Used

- Big-O is the most commonly used notation in practice, because worst-case guarantees are most useful for reliability

Common Time Complexities & Growth Rate

Notation	Name	Example
$O(1)$	Constant	Array access by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search / single loop
$O(n \log n)$	Linearithmic	Merge sort, quick sort
$O(n^2)$	Quadratic	Bubble sort, nested loops
$O(2^n)$	Exponential	Recursive Fibonacci (naive)



Best to worst (typical order): $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

Space Complexity

- **Definition**

- Space complexity is the total amount of memory space an algorithm needs to run to completion, as a function of input size n .

- **Formula**

- $S(P) = C + S_p(\text{instance characteristics})$

- **Fixed Part (C)**

- Constant space required for: simple variables, fixed-size constants, program code / instructions — independent of input size.

- **Variable Part (S_p)**

- Space that depends on the particular problem instance: dynamically allocated memory, recursion stack space, variable-size data structures.



Example

- An array of n integers needs roughly $4n$ bytes $\rightarrow$ Space Complexity = $O(n)$
- A simple variable swap needs constant extra space $\rightarrow O(1)$

Example: Calculating Time Complexity

```
sum = 0; // 1 operation

for (i = 0; i < n; i++) {
    sum = sum + i; // runs n times
}

return sum; // 1 operation
```



Step-by-Step

- Statement 1 (sum = 0) executes ONCE → constant time.
- The loop runs n times, and the statement inside it also executes n times.
- Statement 'return sum' executes ONCE → constant time.
- Total steps $\approx 1 + n + 1 = n + 2$
- Dropping constants (as per Big-O rules) → Time Complexity = $O(n)$

Key Takeaways

- ✓ A Data Structure organizes data + relationships + operations for efficient use
- ✓ Primitive vs Non-Primitive → Non-Primitive splits into Linear & Non-Linear
- ✓ Six core operations: Traverse, Insert, Delete, Search, Sort, Merge
- ✓ Arrays give $O(1)$ random access via contiguous memory & address formulas
- ✓ Structures group different types; Unions share memory; Self-referential structures enable linked lists
- ✓ Pointers + malloc/calloc/realloc/free enable dynamic, runtime memory management
- ✓ Performance = Time Complexity (Big-O, Omega, Theta) + Space Complexity

Thank You

Questions & Discussion